

DOCUMENTACIÓN — CHAPACO SPORT

INDICE

1. Arquitectura general.....	1
1.1. Tecnologías detectadas.....	1
1.2. Capas reales.....	1
1.3. Flujo global de datos.....	2
2. Arranque del sistema, explicado por bloques.....	2
2.1. index.html.....	2
2.2. src/main.jsx.....	3
2.3. src/App.jsx.....	3
3. Rutas completas.....	3
3.1. Rutas cliente — src/rutas/RutasCliente.jsx.....	3
3.2. Rutas Admin — src/rutas/RutasAdmin.jsx.....	4
4. Firebase y configuración central.....	4
4.1. src/servicios/firebaseConfig.js.....	4
4.2. Persistencia local de Firestore	5
5. MODELO DE DATOS EN CLOUD FIRESTORE.....	5
5.1. Colección anuncios_barra.....	2
5.3. Colección categorías.....	2
5.4. Colección codigos_descuento.....	3
5.5. Colección configuracion_negocio.....	3
5.6. Colección marcas.....	4
5.7. Colección packs_ofertas.....	4
5.8. Colección productos.....	6
5.9. Colección redes_sociales.....	7
5.10. Colección roles.....	7
5.11. Colección ubicaciones.....	8
5.12. Colección usuarios	8
5.13. Colección whatsapps	1
6. Autenticación, Sesión y Permisos.....	1

6.1. Inicio de sesión	1
6.2. Persistencia de sesión.....	1
6.3. Resolución de permisos.....	2
6.4. Seguridad de Firestore y diferencia respecto a los permisos visuales.....	2
7. Carrito y pedido por WhatsApp	3
7.1. Estado del carrito	3
7.3. Agregar un producto.....	4
7.3. Finalizar pedido	4
8. Catálogo público	4
8.1. Catálogo	4
8.2. Búsqueda.....	1
8.3. Ofertas	1
8.4. Marcas.....	1
8.5. Promociones.....	1
9. Detalle de producto y variantes.....	2
10. Administración de productos	2
10.1. Listado	2
10.2. Crear producto	3
10.3. Editar producto.....	3
10.4. Variantes.....	3
11. Módulo de promociones, packs y códigos de descuento	3
11.1. Acceso administrativo y archivos principales.....	3
11.2. Packs automáticos desde el panel Admin.....	2
11.3. Publicación y consulta de packs por el cliente.....	3
11.4. Aplicación automática de packs en el carrito.....	3
11.5. Administración de códigos de descuento.....	4
11.6. Validación y aplicación del código en el carrito	4
11.7. Integración del descuento con el pedido de WhatsApp	5
11.8. Archivos que componen el módulo	5
12. Contenido administrable.....	6
12.1. Barra de anuncios	6
12.2. Carrusel.....	6

12.3. Configuración del negocio	6
13. SEO y build	6
13.1. SEO mientras el usuario navega	6
13.2. SEO generado durante el build	1
13.3. vite.config.js	1
13.4. firebase.json	1
14. RESUMEN DE FUNCIONALIDADES DEL SISTEMA	2
17. Organización de carpetas del proyecto	3
18. Guía práctica para modificar el proyecto	3
21. Mapa general y organización técnica del proyecto	6
23. Conclusión para incorporación de un desarrollador nuevo	7
PARTE II — INVENTARIO TÉCNICO AMPLIADO Y TRAZABILIDAD DE CÓDIGO	8
24. Configuración de raíz del proyecto	8

PARTE I — DESCRIPCIÓN FUNCIONAL Y ARQUITECTÓNICA DETALLADA

Documento construido exclusivamente a partir del código fuente, archivos de configuración y estructura real del proyecto analizado. Cuando una característica no puede demostrarse desde el repositorio, se indica explícitamente. No se documentan autores, responsables ni historial de cambios.

0. Cómo leer este documento

Esta documentación está orientada a mantenimiento. Para cada módulo se explica qué archivo inicia el flujo, qué dependencias consume, qué estado maneja, qué servicio consulta o modifica datos y qué colección de Firebase termina involucrada. Las rutas se escriben exactamente como existen en el proyecto. Los archivos vacíos también se señalan porque son relevantes para evitar que un desarrollador busque lógica donde actualmente no existe.

El sistema **no contiene un backend propio con API REST, controllers o repositories**. La aplicación React se comunica directamente con Firebase Authentication, Cloud Firestore y Firebase Storage mediante el SDK de Firebase. Por tanto, cuando en una arquitectura tradicional se esperaría Página → API → Controller → Service → Repository → DB, en este proyecto el flujo real suele ser Página/Componente → Servicio o Contexto → SDK Firebase → Firestore/Storage/Auth.

1. Arquitectura general

1.1. Tecnologías detectadas

Área	Tecnología real	Uso
UI	React 19	Renderizado de páginas y componentes
Enrutamiento	react-router-dom 7	Rutas cliente y administrativas
Build	Vite 8	Desarrollo, empaquetado y división de chunks
Autenticación	Firebase Authentication	Inicio/cierre de sesión y restablecimiento de contraseña
Datos	Cloud Firestore	Catálogo, usuarios, roles, configuración y contenido administrable
Archivos	Firebase Storage	Imágenes de productos, carrusel, logos y guías
Analítica	Firebase Analytics	Inicialización condicional cuando el navegador la soporta
Alertas	SweetAlert2	Confirmaciones y mensajes principalmente en Admin
Iconos	lucide-react / react-icons	Iconografía UI
SEO	Componente propio + script Node	Metadatos dinámicos y páginas estáticas de productos para build
Despliegue	Firebase Hosting	Publicación de dist y rewrite SPA

1.2. Capas reales

1. **Entrada y bootstrap:** index.html → src/main.jsx → src/App.jsx.

2. **Enrutamiento:** src/rutas/RutasCliente.jsx y src/rutas/RutasAdmin.jsx.
3. **Layouts:** src/componentes/cliente/LayoutCliente.jsx y src/componentes/admin/LayoutAdmin.jsx.
4. **Páginas:** src/paginas/cliente/* y src/paginas/admin/*.
5. **Componentes reutilizables:** src/componentes/*.
6. **Contextos/Hooks:** estado compartido de carrito, catálogo y permisos. Ver src/contextos/* y src/hooks/*.
7. **Servicios:** acceso a Firebase y lógica de persistencia. Ver src/servicios/admin/*, src/servicios/cliente/* y src/servicios/firebaseConfig.js.
8. **Persistencia:** Firebase Authentication, Firestore y Storage. No existe capa backend propia. El SDK se ejecuta en el navegador.
9. **Utilidades:** constantes, formateo, carga de datos de prueba y helpers varios en src/utilidades/*.
10. **Build/SEO:** vite.config.js, scripts/generar-seo-productos.mjs, public/robots.txt y public/sitemap.xml.

1.3. Flujo global de datos

Navegador

→ index.html
 → src/main.jsx
 → ProveedorCarrito
 → src/App.jsx
 → BrowserRouter
 | — /admin/* → RutasAdmin → RutaPrivada → ProveedorPermisosAdmin → LayoutAdmin → páginas

Admin

| — /* → RutasCliente → LayoutCliente → ProveedorCatalogoPublico → páginas Cliente

Página/Componente

→ Hook/Contexto o Servicio
 → src/servicios/firebaseConfig.js
 → SDK Firebase
 → Authentication / Firestore / Storage
 → resultado/snapshot
 → estado React
 → renderizado UI

2. Arranque del sistema, explicado por bloques

2.1. index.html

Es el documento HTML base que Vite sirve en desarrollo y usa como plantilla para producción. Contiene el nodo raíz donde React se monta. Su función no es implementar lógica de negocio; su responsabilidad

es entregar el HTML inicial y metadatos base. `src/main.jsx` es el código que toma ese contenedor y monta la aplicación.

Relación: `index.html` → contiene el contenedor raíz → `src/main.jsx` → monta React dentro de ese contenedor.

2.2. `src/main.jsx`

Este archivo realiza cuatro acciones concretas: importa `StrictMode`, obtiene `createRoot`, carga `src/index.css`, importa `App` y envuelve toda la aplicación en `ProveedorCarrito`. Por ello, cualquier componente descendiente puede consumir el carrito mediante `usarCarrito()` sin volver a crear el estado.

Relación: `src/main.jsx` → envuelve → `src/contextos/ContextoCarrito.jsx` → expone estado y operaciones del carrito a toda la app.

Relación: `src/main.jsx` → renderiza → `src/App.jsx` → decide si la URL pertenece al área cliente o Admin.

2.3. `src/App.jsx`

`src/App.jsx` crea `BrowserRouter`, coloca un `Suspense` global y registra dos ramas principales: `/admin/*` y `/*`. `RutasAdmin` se carga con `lazy()`, por lo que el código administrativo queda fuera del bundle inicial del cliente. `RutasCliente` se importa de manera directa y contiene las rutas públicas. El fallback de carga es `src/componentes/comun/CargandoPagina.jsx`.

Relación: `src/App.jsx` → lazy import → `src/rutas/RutasAdmin.jsx` → reduce código Admin cargado durante navegación pública.

Relación: `src/App.jsx` → usa directamente → `src/rutas/RutasCliente.jsx` → resuelve las pantallas públicas.

3. Rutas completas

3.1. Rutas cliente — `src/rutas/RutasCliente.jsx`

URL	Archivo renderizado	Observación
/	<code>src/paginas/cliente/Inicio.jsx</code>	Inicio
/catalogo	<code>src/paginas/cliente/Catalogo.jsx</code>	Catálogo con filtros/búsqueda
/marcas	<code>src/paginas/cliente/Marcas.jsx</code>	Navegación por marcas
/busqueda	<code>src/paginas/cliente/Catalogo.jsx</code>	Reutiliza el catálogo para resultados de búsqueda
/ofertas	<code>src/paginas/cliente/Ofertas.jsx</code>	Productos en oferta
/promociones	<code>src/paginas/cliente/Promociones.jsx</code>	Packs y promociones activas
/producto/:idProducto	<code>src/paginas/cliente/DetalleProducto.jsx</code>	Detalle y selección de variantes
cualquier otra	<code>src/paginas/cliente/PaginaNoEncontrada.jsx</code>	404 visual del cliente

Todas estas rutas están anidadas bajo `src/componentes/cliente/LayoutCliente.jsx`, por lo que comparten barra de anuncios, cabecera, pie de página y proveedor del catálogo.

3.2. Rutas Admin — `src/rutas/RutasAdmin.jsx`

URL	Archivo	Protección
<code>/admin</code>	<code>src/paginas/admin/IniciarSesion.jsx</code>	Pública, para login
<code>/admin/panel</code>	<code>src/paginas/admin/GestionProductos.jsx</code>	<code>productos_ver</code>
URL	Archivo	Protección
<code>/admin/productos/nuevo</code>	<code>src/componentes/admin/FormularioProducto.jsx</code>	<code>productos_crear</code>
<code>/admin/productos/editar/:idProducto</code>	<code>src/componentes/admin/FormularioProducto.jsx</code>	<code>productos_editar</code>
<code>/admin/categorias</code>	<code>src/paginas/admin/GestionCategorias.jsx</code>	<code>categorias_gestionar</code>
<code>/admin/marcas</code>	<code>src/paginas/admin/GestionMarcas.jsx</code>	<code>marcas_gestionar</code>
<code>/admin/usuarios</code>	<code>src/paginas/admin/GestionUsuarios.jsx</code>	<code>usuarios_gestionar</code>
<code>/admin/roles</code>	<code>src/paginas/admin/GestionRoles.jsx</code>	<code>roles_gestionar</code>
<code>/admin/carrusel</code>	<code>src/paginas/admin/GestionCarrusel.jsx</code>	<code>carrusel_gestionar</code>
<code>/admin/anuncios</code>	<code>src/paginas/admin/GestionAnunciosBarra.jsx</code>	<code>anuncios_gestionar</code>
<code>/admin/promociones</code>	<code>src/paginas/admin/GestionPromociones.jsx</code>	<code>promociones_gestionar</code>
<code>/admin/configuracionnegocio</code>	<code>src/paginas/admin/GestionConfiguracionNegocio.jsx</code>	<code>configuracion_negocio_gestionar</code>

Antes de renderizar cualquier ruta Admin autenticada, `src/rutas/RutaPrivada.jsx` comprueba Firebase Authentication. Después `src/contextos/ContextoPermisosAdmin.jsx` obtiene el documento de usuarios asociado al UID, carga su documento `roles/{roleId}` y `src/rutas/RutaPermiso.jsx` decide si puede entrar a cada ruta.

`BloqueoIndexacionAdmin`, definido en `src/rutas/RutasAdmin.jsx`, inserta o modifica `meta[name="robots"]` con `noindex`, `nofollow`, `noarchive` mientras el árbol Admin está montado. Al desmontarse restaura el valor anterior.

4. Firebase y configuración central

4.1. `src/servicios/firebaseConfig.js`

Este archivo es el punto común de acceso a Firebase. Lee siete variables `VITE_FIREBASE_*`, inicializa la aplicación, exporta `auth`, crea Firestore con caché local persistente para múltiples pestañas, exporta `storage` e intenta activar Analytics solo si `isSupported()` devuelve verdadero.

Variables requeridas por el código:

```
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
VITE_FIREBASE_MEASUREMENT_ID=
```

Relación: cualquier servicio que importa db → depende de Firestore inicializado en src/servicios/firebaseConfig.js.

Relación: src/rutas/RutaPrivada.jsx y src/servicios/admin/authServicio.js → dependen de auth/app → Firebase Authentication.

Relación: src/servicios/admin/storageServicio.js → depende de storage → Firebase Storage.

4.2. Persistencia local de Firestore

initializeFirestore(... persistentLocalCache({ tabManager: persistentMultipleTabManager() }))) solicita caché persistente y coordinación entre pestañas. Esto significa que Firestore puede reutilizar datos cacheados en el navegador. No debe confundirse con un modo offline de negocio completamente diseñado: el código demuestra caché persistente del SDK, pero no existe una capa propia de cola/sincronización documentada en el repositorio.

5. MODELO DE DATOS EN CLOUD FIRESTORE

Chapaco Sport utiliza **Cloud Firestore** como base de datos documental. A diferencia de una base SQL, no existen tablas con un esquema rígido: la información se organiza en **colecciones**, cada colección contiene **documentos** y cada documento almacena campos con distintos tipos de datos.

La estructura presentada en esta sección corresponde a las colecciones y campos comprobados en la base de datos y en el código actual del sistema.

Actualmente se observan las siguientes colecciones:

- anuncios_barra
- carrusel
- categorías
- codigos_descuento
- configuracion_negocio
- marcas
- packs_ofertas
- productos
- redes_sociales
- roles
- ubicaciones
- usuarios
- whatsapps

Los tipos empleados en las siguientes tablas corresponden a los tipos de Firestore:

- string: texto.
- boolean: true o false.
- number: número entero o decimal.
- timestamp: fecha y hora almacenada mediante Firestore.

- array: lista de valores.
- map: objeto que contiene otros campos.

5.1. Colección anuncios_barra

Almacena los mensajes que se muestran en la barra superior del sitio público. Cada documento representa un anuncio independiente.

Campo	Tipo	Descripción
activo	boolean	Determina si el anuncio debe mostrarse al cliente.
orden	number	Define la posición del anuncio respecto a los demás.
texto	string	Contenido visible del anuncio.

Flujo relacionado:

- Firestore/anuncios_barra
- src/servicios/cliente/barraAnunciosPublicoServicio.js
- src/hooks/usarAnunciosBarra.js
- src/componentes/cliente/BarraAnuncios.jsx
- barra superior del sitio

La administración se realiza mediante:

- src/paginas/admin/GestionAnunciosBarra.jsx
- src/servicios/admin/anunciosBarraServicio.js
- Firestore/anuncios_barra

5.3. Colección categorías

Contiene las categorías utilizadas para clasificar productos.

Campo	Tipo	Descripción
descripcion	string	Descripción de la categoría.
estado	string	Estado observado en los documentos actuales. En la captura se encuentra almacenado como texto, por ejemplo "true".
nombre	string	Nombre visible de la categoría.
guia_tallas_url	string, opcional	URL pública de la guía de tallas cuando la categoría posee una.
guia_tallas_ruta	string, opcional	Ruta del archivo correspondiente dentro de Storage.

Relación con productos:

productos.categoria_id

- ID de documento en categorías

5.4. Colección `codigos_descuento`

Almacena los códigos promocionales que pueden ser introducidos por el cliente durante la compra.

Campo	Tipo	Descripción
activo	boolean	Indica si el código puede utilizarse.
actualizado_en	timestamp	Fecha y hora de la última actualización.
codigo	string	Código que escribe el cliente.
combinable_con_packs	boolean	Indica si el código puede utilizarse conjuntamente con un pack.
compra_minima	number	Importe mínimo requerido para aplicar el descuento.
creado_en	timestamp	Fecha y hora de creación.
descripcion	string	Descripción administrativa de la promoción.
descuento_maximo	number	Máximo descuento permitido para el código.
es_prueba	boolean	Identifica registros creados para pruebas.
fecha_fin	string	Fecha final de vigencia. Puede estar vacía cuando la vigencia es indefinida.
fecha_inicio	string	Fecha inicial de vigencia.
nombre	string	Nombre descriptivo de la campaña o código.
porcentaje_descuento	number	Porcentaje de descuento aplicado.
vigencia_indefinida	boolean	Indica si el código no posee fecha de finalización obligatoria.

5.5. Colección `configuracion_negocio`

Contiene configuración general reutilizada por el sitio público.

Campo	Tipo	Descripción
actualizado_en	timestamp	Fecha y hora de última modificación.
descripcion_footer	string	Texto descriptivo mostrado en el pie de página.

Flujo: Administración

- GestionConfiguracionNegocio.jsx
- configuracionNegocioServicio.js
- Firestore/configuracion_negocio
- configuracionNegocioClienteServicio.js
- PieDePagina.jsx

5.6. Colección marcas

Almacena las marcas de los productos.

Campo	Tipo	Descripción
estado	boolean	Determina si la marca está activa.
logo	string	URL pública del logo almacenado en Firebase Storage.
logoRuta	string	Ruta física/lógica del archivo dentro de Storage.
nombre	string	Nombre comercial de la marca.

Relación: productos.marca_id

- ID de documento en marcas

Ejemplo conceptual: productos.marca_id = "gymshark"

- marcas/gymshark

5.7. Colección packs_ofertas

Representa promociones que agrupan productos según determinadas condiciones.

Campo	Tipo	Descripción
activo	boolean	Determina si el pack está habilitado.
actualizado_en	timestamp	Última modificación.

cantidad_minima	number	Cantidad mínima de artículos que debe cumplir la selección.
cantidad_paga	number	Cantidad utilizada por determinadas modalidades promocionales.
condiciones	map	Agrupar las restricciones que determinan qué productos participan.
creado_en	timestamp	Fecha de creación.
descripcion	string	Explicación visible o administrativa del pack.
es_prueba	boolean	Indica si corresponde a una promoción utilizada para pruebas.
fecha_fin	string	Fecha de finalización cuando existe vigencia limitada.
fecha_inicio	string	Fecha de inicio.
modalidad	string	Tipo de promoción; por ejemplo precio_fijo_pack.
nombre	string	Nombre del pack.
porcentaje_descuento	number	Porcentaje utilizado cuando la modalidad trabaja con descuento porcentual.
precio_fijo_pack	number	Precio total definido cuando el pack utiliza precio fijo.
repetible	boolean	Indica si el beneficio puede aplicarse más de una vez en el mismo carrito.
requiere_todos_productos	boolean	Indica si deben cumplirse todos los productos establecidos.
vigencia_indefinida	boolean	Indica si no se requiere fecha de finalización.

Estructura de condiciones: condiciones es un map compuesto por:

Campo interno	Tipo	Descripción
categorias_ids	array	IDs de categorías admitidas.
colores	array	Colores admitidos.

marcas_ids	array	IDs de marcas admitidas.
productos_ids	array	IDs concretos de productos incluidos.
tallas	array	Tallas admitidas por la promoción.

La promoción puede utilizar uno o varios de estos criterios dependiendo de su configuración.

5.8. Colección productos

Es la colección principal del catálogo. Cada documento representa un producto.

Campo	Tipo	Descripción
categoria_id	string	ID de la categoría asociada.
descripcion_corta	string	Descripción resumida del producto.
es_nuevo	boolean	Marca el producto como nuevo o parte de un nuevo drop.
estado	string	Estado general del producto, por ejemplo activo.
imagenes	array	Lista de URLs públicas de imágenes.
marca_id	string	ID de la marca.
nombre	string	Nombre comercial del producto.
precio	number	Precio base.
subcolecciones	map	Mapa interno que actualmente contiene las variantes.

Aunque el campo se denomina subcolecciones, las variantes observadas están almacenadas **dentro del propio documento** como mapas.

Estructura:

producto

--> subcolecciones --> variantes

Campos observados en cada variante

Campo	Tipo	Descripción
-------	------	-------------

cod_producto	string	Código/SKU asociado a la variante.
color_estampado	string	Color o descripción visual de la combinación.
disponible	boolean	Indica si la variante puede venderse.
en_oferta	boolean	Determina si posee oferta específica.
nombre_variante	string	Nombre generado de la combinación.
talla	string	Talla de la variante.
precio_oferta	number, opcional	Precio promocional cuando la variante se encuentra en oferta.

5.9. Colección redes_sociales

Campo	Tipo	Descripción
activo	boolean	Define si la red debe mostrarse.
actualizado_en	timestamp	Última actualización.
nombre	string	Nombre visible.
orden	number	Posición visual.
tipo	string	Identificador del tipo de red, por ejemplo tiktok.
url	string	Dirección externa de la cuenta.

5.10. Colección roles

Campo	Tipo	Descripción
estado	boolean	Indica si el rol se encuentra habilitado.
nombre	string	Nombre visible del rol.
permisos	map	Conjunto de permisos booleanos del rol.

Mapa permisos

Los permisos observados incluyen:

Permiso	Tipo
anuncios_gestionar	boolean
carrusel_gestionar	boolean
categorias_gestionar	boolean
configuracion_negocio_gestionar	boolean
marcas_gestionar	boolean

productos_crear	boolean
productos_editar	boolean
productos_eliminar	boolean
productos_ver	boolean
promociones_gestionar	boolean
roles_gestionar	boolean
usuarios_gestionar	boolean

Estos valores son consumidos por el panel administrativo para decidir qué módulos y operaciones puede utilizar cada usuario.

5.11. Colección ubicaciones

Almacena sucursales o ubicaciones físicas.

Campo	Tipo	Descripción
activo	boolean	Indica si se muestra la ubicación.
actualizado_en	timestamp	Última modificación.
direccion	string	Dirección textual.
es_principal	boolean	Identifica la ubicación principal.
nombre	string	Nombre de la sucursal o ubicación.
orden	number	Orden visual.
url_iframe	string	URL utilizada para incrustar el mapa.
url_mapa	string	URL normal de Google Maps u otro proveedor.

5.12. Colección usuarios

Contiene el perfil administrativo asociado a las cuentas autenticadas mediante Firebase Authentication.

Campo	Tipo	Descripción
contraseña	string	Campo observado actualmente en Firestore. Contiene información sensible y no debe utilizarse como sustituto de Firebase Authentication.
correo	string	Correo asociado al usuario.
estado	boolean	Determina si el perfil administrativo está habilitado.
nombre	string	Nombre visible.
rol	string	ID del documento correspondiente en roles.
uid	string	UID generado por Firebase Authentication.

Relaciones: Firebase Authentication user.uid

- usuarios.uid
- usuarios.rol
- roles/{roleId}
- permisos

5.13. Colección whatsapps

Contiene los operadores o números utilizados para la finalización de pedidos.

Campo	Tipo	Descripción
activo	boolean	Indica si el operador puede recibir pedidos.
actualizado_en	timestamp	Fecha de última modificación.
etiqueta	string	Etiqueta funcional, por ejemplo Ventas.
nombre	string	Nombre descriptivo del operador.
numero	string	Número utilizado para construir la URL de WhatsApp.
orden	number	Prioridad u orden de presentación.

Flujo: Firestore/whatsapps

- operadoresWhatsappClienteServicio.js
- whatsappServicio.js
- MiCarrito.jsx
- https://wa.me/...

6. Autenticación, Sesión y Permisos

6.1. Inicio de sesión

El acceso administrativo utiliza Firebase Authentication y, adicionalmente, un perfil almacenado en la colección usuarios.

Usuario

- ➔ /admin
- ➔ src/paginas/admin/IniciarSesion.jsx
- ➔ iniciarSesion(correo, contraseña)
- ➔ src/servicios/admin/authServicio.js
- ➔ Firebase Authentication
- ➔ signInWithEmailAndPassword()
- ➔ Firebase devuelve user.uid
- ➔ consulta Firestore/usuarios
- ➔ busca documento cuyo uid coincide
- ➔ valida usuarios.estado
- ➔ permite o rechaza el acceso

Firebase Authentication valida las credenciales, mientras que Firestore determina si esa cuenta posee un perfil administrativo activo.

6.2. Persistencia de sesión

No se mantiene manualmente un token propio del proyecto.

Firestore Authentication

- conserva el estado de autenticación
- `onAuthStateChanged()`
- `src/rutas/RutaPrivada.jsx`
- comprueba si existe usuario autenticado
- permite cargar rutas administrativas

`src/contextos/ContextoPermisosAdmin.jsx` también observa la sesión para recuperar el perfil y los permisos correspondientes.

6.3. Resolución de permisos

Firestore Auth `user.uid`

- buscar usuarios donde `uid == user.uid`
- obtener `usuarios.rol`
- abrir `roles/{rol}`
- leer `roles.permisos`
- `src/contextos/ContextoPermisosAdmin.jsx`
- `usarPermisosAdmin()`
- `RutaPermiso.jsx` / `BarraLateralAdmin.jsx`
- permitir, bloquear u ocultar funcionalidad

Por ejemplo:

- `productos_editar = true`
 - se permite acceso a edición
- `productos_eliminar = false`
 - la acción de eliminación no debe estar disponible

6.4. Seguridad de Firestore y diferencia respecto a los permisos visuales

La seguridad tiene dos niveles distintos.

Permisos de React: controlan interfaz y navegación

Firestore Security Rules: controlan realmente qué lectura/escritura acepta Firestore

Las reglas actuales de Firestore proporcionadas son:

```
match /{document=**}
```

- lectura pública
- escritura para cualquier usuario autenticado

Además existen reglas específicas para:

anuncios_barra

- ➔ lectura pública
- ➔ crear/editar/eliminar: autenticado

carrusel

- ➔ lectura pública
- ➔ crear/editar/eliminar: autenticado

packs_ofertas

- ➔ lectura pública
- ➔ crear/editar/eliminar: autenticado

codigos_descuento

- ➔ get público
- ➔ list autenticado
- ➔ crear/editar/eliminar: autenticado

7. Carrito y pedido por WhatsApp

7.1. Estado del carrito

src/contextos/ContextoCarrito.jsx mantiene el carrito de forma global.

src/main.jsx

- ➔ ProveedorCarrito
- ➔ ContextoCarrito
- ➔ usarCarrito.js
- ➔ componentes del cliente

El contexto permite: agregar producto

- ➔ modificar cantidad
- ➔ eliminar producto
- ➔ calcular cantidad total
- ➔ calcular subtotal
- ➔ vaciar carrito
- ➔ guardar carrito en almacenamiento local

También escucha cambios del almacenamiento del navegador para mantener coherencia entre pestañas abiertas.

7.3. Agregar un producto

/producto/:idProducto

- DetalleProducto.jsx
- carga producto
- usuario selecciona talla
- usuario selecciona color
- se identifica variante
- se comprueba disponibilidad
- se define cantidad
- manejarAgregarCarrito()
- usarCarrito()
- ContextoCarrito
- almacenamiento local
- Cabecera / ResumenCarrito / MiCarrito

El elemento guardado conserva la información necesaria para diferenciar una combinación concreta de producto, talla y color.

7.3. Finalizar pedido

cliente abre carrito

- MiCarrito.jsx
- completa datos personales y entrega
- validarFormulario()
- consulta operadores activos
- Firestore/whatsapps
- selecciona operador
- whatsappServicio.js
- genera texto del pedido
- genera URL wa.me
- abre WhatsApp

La versión actual no crea una colección pedidos en Firestore como resultado final. El pedido se entrega al negocio mediante WhatsApp.

8. Catálogo público

El catálogo público comparte los productos, categorías y marcas mediante src/contextos/ContextoCatalogoPublico.jsx.

LayoutCliente.jsx

- ProveedorCatalogoPublico
- Firestore/productos
- Firestore/categorias
- Firestore/marcas
- normaliza información
- estado React compartido
- usarCatalogo()
- Catalogo / Marcas / Ofertas / Inicio

8.1. Catálogo

/catalogo

- Catalogo.jsx
- usarCatalogo()
- productos disponibles en memoria
- filtros
- búsqueda
- ordenamiento
- paginación
- TarjetaProducto.jsx

8.2. Búsqueda

La búsqueda se realiza del lado del cliente:

texto escrito

- ➔ normalización
- ➔ calcularPuntajeBusqueda()
- ➔ comparar nombre / marca / categoría / descripción / variantes
- ➔ ordenar coincidencias
- ➔ mostrar resultados

8.3. Ofertas

- ➔ Ofertas.jsx
- ➔ usarCatalogo()
- ➔ productos y variantes
- ➔ detectar ofertas
- ➔ filtrar categoría
- ➔ ordenar
- ➔ TarjetaProducto.jsx

8.4. Marcas

/marcas

- ➔ Marcas.jsx
- ➔ usarCatalogo()
- ➔ seleccionar marca
- ➔ comparar producto.marca_id
- ➔ mostrar productos pertenecientes a la marca

8.5. Promociones

La ruta /promociones presenta los packs activos configurados desde el panel administrativo. La página no mantiene una copia separada de las promociones: se suscribe directamente a Firestore mediante `src/servicios/cliente/promocionesPublicoServicio.js` y reutiliza el catálogo público para resolver marcas, categorías y productos elegibles.

/promociones

- ➔ `src/paginas/cliente/Promociones.jsx`
- ➔ `suscribirPacksActivos()`
- ➔ `Firestore/packs_ofertas`
- ➔ filtra promociones activas y vigentes
- ➔ `src/componentes/cliente/TarjetaPack.jsx`
- ➔ muestra beneficio, condiciones y productos elegibles

Además, `src/paginas/cliente/Inicio.jsx` incorpora `src/componentes/cliente/SeccionPromocionesPacks.jsx`. Esta sección funciona como anuncio de los

packs vigentes: muestra hasta tres promociones y dirige a /promociones para consultar todas las condiciones.

9. Detalle de producto y variantes

La pantalla de detalle concentra la selección final que realiza el cliente antes de agregar un artículo al carrito.

/producto/:idProducto

- ➔ DetalleProducto.jsx
- ➔ obtiene idProducto desde la URL
- ➔ Firestore/productos/{idProducto}
- ➔ obtiene marca_id
- ➔ Firestore/marcas/{marca_id}
- ➔ obtiene categoria_id
- ➔ Firestore/categorias/{categoria_id}
- ➔ renderiza información

Posteriormente: producto

- ➔ subcolecciones.variantes
- ➔ obtener tallas disponibles
- ➔ obtener colores disponibles
- ➔ seleccionar talla/color
- ➔ localizar variante exacta
- ➔ verificar disponible
- ➔ determinar precio
- ➔ agregar al carrito

src/componentes/cliente/GaleriaImágenes.jsx gestiona la presentación de fotografías.

La guía de tallas, cuando existe, se obtiene desde la información asociada a la categoría.

10. Administración de productos

10.1. Listado

/admin/panel

- ➔ GestionProductos.jsx
- ➔ productosServicio.obtenerProductos()
- ➔ Firestore/productos
- ➔ categoriasServicio
- ➔ resolver categorías
- ➔ TablaProductos.jsx

La pantalla permite buscar productos y, dependiendo de permisos, crear, editar o eliminar.

10.2. Crear producto

/admin/productos/nuevo

- ➔ FormularioProducto.jsx
- ➔ carga categorías
- ➔ carga marcas
- ➔ usuario completa información
- ➔ TablaVariantesProducto.jsx
- ➔ usuario agrega talla/color/stock
- ➔ usuario selecciona imágenes
- ➔ validarFormulario()
- ➔ Storage
- ➔ obtiene URLs
- ➔ productosServicio.js
- ➔ Firestore/productos

10.3. Editar producto

/admin/productos/editar/:idProducto

- ➔ FormularioProducto.jsx
- ➔ obtenerProductoPorId()
- ➔ precarga información existente
- ➔ usuario modifica datos / variantes / imágenes
- ➔ elimina archivos sustituidos cuando corresponde
- ➔ actualizarProducto()
- ➔ Firestore + Storage

10.4. Variantes

TablaVariantesProducto.jsx

- ➔ genera combinaciones talla/color
- ➔ genera o administra código
- ➔ disponibilidad
- ➔ oferta de variante
- ➔ entrega mapa de variantes
- ➔ FormularioProducto.jsx
- ➔ productos.subcolecciones.variantes

11. Módulo de promociones, packs y códigos de descuento

El módulo de promociones agrega dos mecanismos distintos de descuento: packs automáticos y códigos escritos por el cliente. Ambos se administran desde una sola pantalla del panel, pero se almacenan en colecciones Firestore diferentes y se validan por caminos distintos en el cliente.

11.1. Acceso administrativo y archivos principales

/admin/promociones

- ➔ src/rutas/RutasAdmin.jsx
- ➔ src/rutas/RutaPrivada.jsx
- ➔ src/rutas/RutaPermiso.jsx con permiso promociones_gestionar
- ➔ src/paginas/admin/GestionPromociones.jsx
- ➔ src/servicios/admin/promocionesServicio.js
- ➔ Firestore/packs_ofertas y Firestore/codigos_descuento

src/componentes/admin/BarraLateralAdmin.jsx incluye la opción "Promociones" y solo la muestra cuando el rol posee promociones_gestionar. La definición y compatibilidad de este permiso se encuentran en src/servicios/admin/rolesServicio.js.

La pantalla src/paginas/admin/GestionPromociones.jsx utiliza dos pestañas: Packs automáticos y Códigos de descuento. También se suscribe a productos, marcas y categorías para construir selectores usando datos reales del catálogo.

11.2. Packs automáticos desde el panel Admin

Un pack define qué artículos pueden participar, cuántas unidades debe reunir el carrito y qué beneficio obtiene el cliente. GestionPromociones.jsx permite filtrar participantes por marca, categoría, productos específicos, talla y color/estampado. Los filtros seleccionados se cumplen conjuntamente; si no se define ningún filtro, el pack puede aplicar a todo el catálogo.

Modalidad interna	Configuración	Comportamiento
porcentaje_desde	cantidad_minima + porcentaje_descuento	Desde la cantidad mínima, aplica el porcentaje a todas las unidades elegibles del carrito.
porcentaje_pack	cantidad_minima + porcentaje_descuento	Aplica el porcentaje solamente al grupo de prendas definido por el pack.
precio_fijo_pack	cantidad_minima + precio_fijo_pack	Convierte el grupo elegible en un pack con precio total fijo.
lleva_paga	cantidad_minima + cantidad_paga	El cliente lleva X unidades y paga únicamente Y dentro del grupo calculado.

Opciones adicionales del pack: repetible permite aplicar el beneficio por cada grupo completo; requiere_todos_productos exige al menos una unidad de cada producto seleccionado; vigencia_indefinida permite omitir fechas; activo determina si la promoción debe considerarse vigente.

Validaciones principales implementadas en GestionPromociones.jsx: nombre y cantidad mínima obligatorios; porcentajes entre 1 y 90; precio fijo mayor a cero cuando corresponde; cantidad_paga menor a cantidad_minima; productos obligatorios coherentes con el tamaño del pack; y fecha inicial no posterior a la fecha final.

Formulario de pack

- ➔ validarPack()
- ➔ guardarPack()
- ➔ src/servicios/admin/promocionesServicio.js
- ➔ setDoc() sobre Firestore/packs_ofertas/{id}
- ➔ actualizado_en = serverTimestamp()
- ➔ creado_en = serverTimestamp() al crear

Cuando es un pack nuevo, promocionesServicio.js genera un ID a partir del nombre normalizado y un sufijo temporal. La edición conserva el ID existente. eliminarPack() elimina directamente el documento correspondiente.

11.3. Publicación y consulta de packs por el cliente

src/servicios/cliente/promocionesPublicoServicio.js expone suscribirPacksActivos(). La función escucha packs_ofertas en tiempo real, descarta promociones inactivas o fuera de fecha mediante promocionVigente() y ordena los resultados antes de entregarlos a la interfaz.

Firestore/packs_ofertas

- ➔ suscribirPacksActivos()
- ➔ promocionVigente()
- ➔ src/paginas/cliente/Promociones.jsx
- ➔ src/componentes/cliente/TarjetaPack.jsx
- ➔ cliente consulta beneficio y condiciones

TarjetaPack.jsx traduce los IDs de marcas y categorías a nombres visibles, describe tallas y colores admitidos y puede desplegar los productos compatibles. Para determinar esos productos usa productoCumpleFiltrosPack() de src/utilidades/descuentos.js, que revisa filtros del producto y, cuando hay restricciones de talla o color, exige que exista al menos una variante disponible que las cumpla.

En Inicio.jsx, SeccionPromocionesPacks.jsx funciona como bloque de difusión: informa cuántas promociones vigentes existen, muestra hasta tres tarjetas compactas y ofrece acceso a la página completa /promociones.

11.4. Aplicación automática de packs en el carrito

El cliente no debe introducir ningún código para activar un pack. MiCarrito.jsx mantiene una suscripción a los packs activos y recalcula los beneficios cada vez que cambia el carrito o la lista de promociones.

itemsCarrito + packs activos

- ➔ calcularResumenDescuentos()
- ➔ calcularMejorPack()
- ➔ evaluar cada pack vigente
- ➔ itemCumplePack() filtra cada unidad según producto/marca/categoría/talla/color
- ➔ calcularPack() obtiene el ahorro de la modalidad
- ➔ se conserva únicamente el resultado con mayor descuento
- ➔ MiCarrito.jsx muestra promoción aplicada y ahorro

src/utilidades/descuentos.js expande las cantidades del carrito a unidades individuales para calcular correctamente packs repetibles y grupos. Cuando requiere_todos_productos es verdadero, la selección garantiza la presencia de los productos obligatorios antes de completar el resto del grupo.

Si todavía no existe un pack aplicable, obtenerProgresoPack() busca la promoción más cercana entre aquellas donde el carrito ya contiene al menos un artículo elegible. MiCarrito.jsx puede mostrar cuántas prendas faltan para activar ese beneficio.

11.5. Administración de códigos de descuento

La segunda pestaña de GestionPromociones.jsx administra códigos que el cliente escribe manualmente en el carrito. Los documentos se almacenan en `codigos_descuento` y el ID del documento es el propio código normalizado.

Campo funcional	Uso en el módulo
<code>codigo</code>	Texto que introduce el cliente. <code>normalizarCodigoDescuento()</code> lo convierte a mayúsculas, elimina espacios y admite A-Z, 0-9, guion y guion bajo.
<code>porcentaje_descuento</code>	Porcentaje aplicado cuando el código es válido. El formulario acepta valores entre 1 y 90.
<code>compra_minima</code>	Subtotal mínimo requerido para utilizar el código.
<code>descuento_maximo</code>	Tope monetario del descuento. Un valor 0 significa sin límite.
<code>combinable_con_packs</code>	Define si el código puede aplicarse cuando ya existe un pack automático.
<code>fecha_inicio / fecha_fin</code>	Delimitan la vigencia cuando <code>vigencia_indefinida</code> no está activa.
<code>activo</code>	Permite habilitar o deshabilitar el código sin eliminarlo.

Formulario de código

- ➔ `normalizarCodigoDescuento()`
- ➔ `guardarCodigo()`
- ➔ documento `Firestore/codigos_descuento/{CODIGO}`
- ➔ si cambia el código, se crea el nuevo documento y se elimina el ID anterior

`eliminarCodigo()` elimina el documento del código seleccionado. La lista administrativa también permite copiar el código al portapapeles y muestra su estado, porcentaje, compra mínima, compatibilidad con packs y vigencia.

11.6. Validación y aplicación del código en el carrito

`MiCarrito.jsx` incorpora un campo "Código de descuento". El cliente escribe el código y selecciona APLICAR. La consulta no descarga toda la colección pública: `obtenerCodigoDescuento()` normaliza el texto y ejecuta `getDoc()` únicamente sobre `codigos_descuento/{codigo}`.

Cliente escribe código

- ➔ `MiCarrito.jsx`
- ➔ `obtenerCodigoDescuento(texto)`
- ➔ `normalizarCodigoDescuento()`
- ➔ `getDoc(Firestore/codigos_descuento/{CODIGO})`
- ➔ `calcularResumenDescuentos()`
- ➔ `validarCodigo()`
- ➔ comprobar activo/vigencia
- ➔ comprobar `compra_minima`
- ➔ comprobar `combinable_con_packs` cuando existe pack

- ➔ calcular descuento
- ➔ mostrar resultado en el carrito

El descuento del código se calcula sobre el subtotal restante después del pack automático. Si descuento_maximo es mayor que cero, el ahorro generado por el porcentaje se limita a ese monto.

La fórmula final implementada por calcularResumenDescuentos() es:

$$\text{total} = \text{subtotal} - \text{descuentoPack} - \text{descuentoCodigo}$$

Si un código aplicado deja de ser válido porque cambia el carrito o aparece una condición incompatible, MiCarrito.jsx retira el código y muestra el mensaje de validación correspondiente.

11.7. Integración del descuento con el pedido de WhatsApp

El resumen calculado no se utiliza únicamente para mostrar el total en pantalla. Al finalizar la compra, MiCarrito.jsx envía resumenPrecios a src/servicios/cliente/whatsappServicio.js.

MiCarrito.jsx

- ➔ abrirPedidoEnWhatsApp(numero, datosCliente, itemsCarrito, resumenPrecios)
- ➔ whatsappServicio.js
- ➔ agrega subtotal
- ➔ agrega nombre y ahorro del pack si existe
- ➔ agrega código y ahorro del cupón si existe
- ➔ agrega TOTAL final
- ➔ genera URL wa.me
- ➔ abre WhatsApp

Después de abrir el pedido, MiCarrito.jsx vacía el carrito, limpia el formulario y elimina del estado el código aplicado.

11.8. Archivos que componen el módulo

Ruta exacta	Responsabilidad dentro del módulo
src/paginas/admin/GestionPromociones.jsx	Pantalla administrativa de packs y códigos; formularios, validaciones, edición, eliminación y selección de condiciones.
src/componentes/admin/SelectorMultipleBuscable.jsx	Selector reutilizable de marcas, categorías, productos, tallas y colores con búsqueda y selección múltiple.
src/servicios/admin/promocionesServicio.js	Suscripciones y persistencia de packs_ofertas y codigos_descuento.
src/paginas/cliente/Promociones.jsx	Página pública que lista promociones vigentes.

src/componentes/cliente/SeccionPromocionesPacks.jsx	Bloque de difusión de packs insertado en la página de Inicio.
src/componentes/cliente/TarjetaPack.jsx	Presenta condiciones, beneficio y productos elegibles de cada pack.
src/servicios/cliente/promocionesPublicoServicio.js	Lectura pública de packs activos y consulta individual de códigos.
src/utilidades/descuentos.js	Reglas de vigencia, elegibilidad, modalidades de pack, selección del mejor pack, progreso, validación de códigos y cálculo final.
src/paginas/cliente/MiCarrito.jsx	Integra packs automáticos, entrada de códigos, mensajes de validación y total descontado.
src/servicios/cliente/whatsappServicio.js	Incluye los descuentos aplicados en el mensaje final de WhatsApp.
src/rutas/RutasAdmin.jsx	Declara /admin/promociones y exige promociones_gestionar.
src/rutas/RutasCliente.jsx	Declara la página pública /promociones.
src/servicios/admin/rolesServicio.js	Declara y normaliza el permiso promociones_gestionar.

12. Contenido administrable

12.1. Barra de anuncios

Permite crear mensajes, cambiar su estado y modificar el orden.

12.2. Carrusel

Permite administrar imágenes, textos, enlaces, estado y orden.

12.3. Configuración del negocio

13. SEO y build

13.1. SEO mientras el usuario navega

src/componentes/comun/SEO.jsx actualiza los datos de la página que está viendo el visitante.

Página React

- | | | |
|-----------|--------------------|--------------|
| ➔ SEO.jsx | ➔ meta description | ➔ Open Graph |
| ➔ <title> | ➔ canonical | ➔ JSON-LD |

Por ejemplo, Inicio, Catálogo, Marcas, Ofertas y Detalle de Producto pueden mostrar metadatos distintos.

13.2. SEO generado durante el build

Cuando se ejecuta:

```
npm run build
```

primero se genera la aplicación con Vite y posteriormente se ejecuta:

```
scripts/generar-seo-productos.mjs
```

El flujo es:

```
npm run build
```

- ➔ Vite genera dist/
- ➔ generar-seo-productos.mjs
- ➔ consulta productos
- ➔ genera HTML destinado a URLs de productos
- ➔ actualiza sitemap correspondiente
- ➔ resultado final queda en dist/

13.3. vite.config.js

Controla cómo se compila la aplicación:

- ➔ Vite
- ➔ división de recursos
- ➔ minificación
- ➔ assets
- ➔ dist/

13.4. firebase.json

Define cómo Firebase Hosting publica el resultado:

- ➔ Firebase Hosting
- ➔ rewrite a /index.html para rutas React
- ➔ políticas de caché
- ➔ headers HTTP
- ➔ sitio público

14. RESUMEN DE FUNCIONALIDADES DEL SISTEMA

Funcionalidad	En qué consiste
Inicio público	Presenta carrusel, accesos promocionales y productos destacados.
Catálogo	Permite explorar, buscar, filtrar y ordenar productos.
Marcas	Agrupar y muestra productos según su marca.
Ofertas	Presenta productos o variantes que poseen precios promocionales.
Detalle de producto	Muestra información, fotografías, variantes, talla, color, precio y disponibilidad.
Carrito	Conserva productos seleccionados, cantidades y subtotales.
Pedido por WhatsApp	Convierte carrito y datos del cliente en un mensaje enviado a un operador configurado.
Login administrativo	Autentica usuarios mediante Firebase Authentication y valida su perfil en Firestore.
Roles y permisos	Controla qué módulos y acciones administrativas aparecen disponibles para cada rol.
Productos	Permite crear, editar, visualizar y eliminar productos y variantes.
Categorías	Organiza los productos y permite asociar guías de tallas.
Marcas	Administra marcas y sus logos.
Barra de anuncios	Permite publicar mensajes breves visibles en todo el sitio.
Carrusel	Administra las diapositivas principales del inicio.
Configuración del negocio	Gestiona footer, redes sociales, ubicaciones y operadores WhatsApp.
Packs de ofertas	Define promociones condicionadas por productos, cantidades, marcas, tallas, colores o categorías.

Códigos de descuento	Permite definir cupones con porcentaje, compra mínima, vigencia y compatibilidad con packs.
----------------------	---

17. Organización de carpetas del proyecto

- `src/componentes/admin/`: UI reutilizable del panel administrativo; sidebar, layout, tabla de productos, formulario y variantes.
- `src/componentes/cliente/`: UI reutilizable pública; header, footer, carrusel, tarjetas, galería, carrito y secciones de inicio.
- `src/componentes/comun/`: componentes transversales como loader, SEO e iconos.
- `src/contextos/`: estado compartido de carrito, catálogo y permisos. `ContextoAutenticacion.jsx` está vacío.
- `src/hooks/`: adaptadores para consumir contextos/servicios; varios hooks son pequeños y algunos placeholders están vacíos.
- `src/paginas/admin/`: pantallas completas de CRUD/configuración administrativa.
- `src/paginas/cliente/`: pantallas completas públicas.
- `src/rutas/`: definición de rutas y guardas de autenticación/permisos.
- `src/servicios/admin/`: operaciones de persistencia administrativas contra Firebase.
- `src/servicios/cliente/`: lectura pública y preparación de WhatsApp.
- `src/utilidades/`: constantes, helpers, datos de prueba y scripts auxiliares ejecutados desde UI.
- `public/`: assets copiados sin transformación al build.
- `scripts/`: scripts Node posteriores/alrededor del build; destaca SEO de productos.
- `dist/`: salida generada; no debe editarse manualmente porque `npm run build` la regenera.

18. Guía práctica para modificar el proyecto

Esta sección indica qué áreas revisar cuando se quiera modificar una funcionalidad, evitando recorrer archivos que no participan en ella.

Agregar una página pública

- Crear la página en: `src/paginas/cliente/`
- Después registrar la ruta en: `src/rutas/RutasCliente.jsx`
- Si debe aparecer en la navegación, revisar: `src/componentes/cliente/Cabecera.jsx`
- Si requiere metadatos propios: `src/componentes/comun/SEO.jsx`

Agregar un módulo administrativo

Crear la pantalla correspondiente dentro de: `src/paginas/admin/`

Luego: nueva pantalla

- ➔ `src/rutas/RutasAdmin.jsx`
- ➔ definir permiso

- ➔ src/servicios/admin/rolesServicio.js
- ➔ RutaPermiso.jsx
- ➔ BarraLateralAdmin.jsx

Modificar información de productos

Revisar conjuntamente:

FormularioProducto.jsx --> captura datos

TablaVariantesProducto.jsx --> tallas / colores / variantes

productosServicio.js --> transforma y persiste

Firestore/productos --> almacena

DetalleProducto.jsx / ContextoCatalogoPublico.jsx --> consume públicamente

Modificar tallas o variantes

Los puntos principales son:

- src/componentes/admin/TablaVariantesProducto.jsx
- src/componentes/admin/FormularioProducto.jsx
- src/servicios/admin/productosServicio.js
- src/paginas/cliente/DetalleProducto.jsx

Un cambio en la estructura de variantes puede afectar talla, color, disponibilidad, precio, oferta y carrito.

Agregar una nueva colección Firestore

Se recomienda: nueva colección

- ➔ crear servicio correspondiente
- ➔ importar db desde firebaseConfig.js
- ➔ implementar lectura/escritura
- ➔ conectar servicio con página o contexto
- ➔ revisar Firebase Security Rules

Modificar roles o permisos

Revisar:

- src/servicios/admin/rolesServicio.js
- src/contextos/ContextoPermisosAdmin.jsx
- src/rutas/RutaPermiso.jsx
- src/rutas/RutasAdmin.jsx
- src/componentes/admin/BarraLateralAdmin.jsx

Modificar contenido del footer

Administración:

- src/paginas/admin/GestionConfiguracionNegocio.jsx
- src/servicios/admin/configuracionNegocioServicio.js
- Cliente:

- src/componentes/cliente/PieDePagina.jsx
- src/servicios/cliente/configuracionNegocioClienteServicio.js

Modificar el pedido de WhatsApp

Revisar:

- src/paginas/cliente/MiCarrito.jsx
- src/servicios/cliente/operadoresWhatsappClienteServicio.js
- src/servicios/cliente/whatsappServicio.js

Modificar promociones, packs o códigos de descuento

Revisar:

src/paginas/admin/GestionPromociones.jsx
 src/servicios/admin/promocionesServicio.js
 src/utilidades/descuentos.js
 src/servicios/cliente/promocionesPublicoServicio.js
 src/paginas/cliente/Promociones.jsx
 src/componentes/cliente/TarjetaPack.jsx
 src/paginas/cliente/MiCarrito.jsx

Si se agrega una nueva condición o modalidad, debe revisarse tanto el formulario administrativo como la función de cálculo del cliente y, si cambia el modelo persistido, las reglas de Firestore correspondientes.

Modificar SEO

- SEO de páginas:
- src/componentes/comun/SEO.jsx
- SEO generado al compilar:
- scripts/generar-seo-productos.mjs
- Archivos públicos relacionados:
- public/robots.txt
- public/sitemap.xml
- Configuración Hosting:
- firebase.json

21. Mapa general y organización técnica del proyecto

```
chapaco-sport/
|
|--> public/
|   |--> recursos públicos
|   |--> robots.txt
|   `--> sitemap.xml
|
|--> scripts/
|   `--> generar-seo-productos.mjs
|
|--> src/
|   |
|   |--> componentes/
|   |   |--> admin/
|   |   |--> cliente/
|   |   `--> comun/
|   |
|   |--> contextos/
|   |   |--> ContextoCarrito.jsx
|   |   |--> ContextoCatalogoPublico.jsx
|   |   `--> ContextoPermisosAdmin.jsx
|   |
|   |--> hooks/
|   |
|   |--> paginas/
|   |   |--> admin/
|   |   `--> cliente/
|   |
|   |--> rutas/
|   |   |--> RutasCliente.jsx
|   |   |--> RutasAdmin.jsx
|   |   |--> RutaPrivada.jsx
|   |   `--> RutaPermiso.jsx
|   |
|   |--> servicios/
|   |   |--> admin/
|   |   |--> cliente/
|   |   `--> firebaseConfig.js
|   |
|   |--> utilidades/
|   |
|   |--> App.jsx
|   `--> main.jsx
|
|--> firebase.json
```

```
|--> vite.config.js
|--> package.json
`--> index.html
```

Flujo general:

```
index.html
--> src/main.jsx
--> src/App.jsx
|
|--> RutasCliente
|   --> LayoutCliente
|   --> páginas públicas
|   --> contextos/hooks
|   --> servicios cliente
|   --> Firebase
|
`--> RutasAdmin
    --> RutaPrivada
    --> ContextoPermisosAdmin
    --> RutaPermiso
    --> LayoutAdmin
    --> páginas administrativas
    --> servicios admin
    --> Firebase
```

Módulo de promociones incorporado:

```
src/paginas/admin/GestionPromociones.jsx
    ➔ src/servicios/admin/promocionesServicio.js
    ➔ Firestore/packs_ofertas + codigos_descuento

src/paginas/cliente/Inicio.jsx
    ➔ SeccionPromocionesPacks.jsx
    ➔ TarjetaPack.jsx

src/paginas/cliente/Promociones.jsx
    ➔ promocionesPublicoServicio.js
    ➔ packs_ofertas

src/paginas/cliente/MiCarrito.jsx
    ➔ promocionesPublicoServicio.js
    ➔ src/utilidades/descuentos.js
    ➔ whatsappServicio.js
```

23. Conclusión para incorporación de un desarrollador nuevo

Para comprender una incidencia, empezar por la URL afectada, localizarla en `src/rutas/RutasCliente.jsx` o `src/rutas/RutasAdmin.jsx`, abrir la página correspondiente y seguir sus imports hacia `hooks/contextos/servicios`. Si el problema es de datos, identificar la colección Firestore indicada en este

documento. Si es de imágenes, seguir hacia `src/servicios/admin/storageServicio.js`. Si es de autorización, seguir `RutaPrivada` → `ContextoPermisosAdmin` → `RutaPermiso`. Si es de SEO/build, seguir `SEO.jsx` → `package.json` → `scripts/generar-seo-productos.mjs` → `firebase.json`.

El principio de mantenimiento más importante es no asumir una arquitectura backend que no existe: en esta versión, el navegador llama directamente a Firebase. Por ello, cualquier cambio de modelo o permiso suele tener impacto simultáneo en UI, servicios cliente/admin y reglas Firebase externas al repositorio.

PARTE II — INVENTARIO TÉCNICO AMPLIADO Y TRAZABILIDAD DE CÓDIGO

24. Configuración de raíz del proyecto

package.json

Define identidad del paquete, scripts de desarrollo/build/lint/preview y dependencias instalables. El script build encadena vite build con `node --env-file=.env scripts/generar-seo-productos.mjs`, por lo que el build de producción incluye una segunda fase SEO que depende de variables de entorno válidas.

vite.config.js

Controla el empaquetado Vite. El código observado fija target: es2020, desactiva source maps de producción, usa minificación oxc, separa CSS, limita inline assets a 4096 bytes, fija warning de chunk en 600 KB y separa manualmente Firebase, React/Router, SweetAlert e iconos.

firebase.json

Configura Firebase Hosting para publicar dist/, reescribir cualquier ruta desconocida hacia `/index.html` para soportar React Router y aplicar políticas de caché/headers. `index.html` y `/producto/**` usan no-cache; `/assets/**` usa caché de un año e immutable; se añaden X-Content-Type-Options y Referrer-Policy.

.firebaserc

Vincula el alias Firebase default con el proyecto chapacosport-1468a. Un deploy ejecutado con esta configuración apunta a ese proyecto salvo que se cambie explícitamente el proyecto activo.

.gitignore

Excluye `node_modules`, `dist`, `logs`, archivos locales y `.env`. Esto evita versionar dependencias generadas y variables de entorno locales.

index.html

Documento HTML base entregado por Vite. Contiene el contenedor de React y metadatos base que posteriormente pueden ser modificados en runtime por `src/componentes/comun/SEO.jsx` o enriquecidos en build por el script SEO.

public/robots.txt

Archivo estático de directivas para rastreadores; se copia al build y Firebase Hosting le aplica caché de una hora.

public/sitemap.xml

Sitemap público base. El script scripts/generar-seo-productos.mjs interviene en la generación/actualización de URLs de producto durante el build.